



Setting Up Full Multigrid

Louis Moresi¹ 

¹Australian National University

Keywords Underworld Code, Tricks of the Trade, development

Most of the effort in a geodynamics model is spent on solving the Stokes equations, and most of that goes into the velocity solver. How that is preconditioned Determines how long a model takes to run, and it is one of the few places where the right choice of settings can be worth an order of magnitude speed-up.

1. WHAT MULTIGRID DOES

An iterative solver reduces error unevenly. Simple relaxation methods (smoothers) are good at removing error that varies rapidly from cell to cell, and poor at removing error that varies smoothly across the whole domain. The smooth part then becomes the expensive part: on a fine mesh it takes many sweeps to move information from one side of the model to the other, and many sweeps to eliminate long-wavelength errors.

Multigrid takes that information and turns it on its head: error that is considered smooth on a fine mesh is *not* so smooth on a mesh twice as coarse, where it spans half as many cells, so a few smoothing sweeps on the coarse mesh can remove errors the fine mesh could not.

The multigrid method is a recursion: smooth on the fine mesh, transfer what is left to a coarser one, solve there (with another recursive step), transfer the correction back, smooth again. Each *level* handles the band of error it can see cheaply, and the work per unknown stops growing as the model gets bigger.

Two things have to be supplied: the coarse levels, and the operators that move between them.

2. TWO WAYS TO BUILD THE COARSE LEVELS

Algebraic multigrid (AMG) builds different resolutions of the problem directly from the matrix. It inspects the operator's connectivity, groups unknowns that are strongly coupled, and constructs coarse levels and transfer operators from that grouping alone. It needs very little from the mesh, and it works everywhere and is the sensible default. PETSc's implementation is GAMG, and it is what Underworld uses when it has nothing better.

Geometric multigrid (GMG) uses actual coarser meshes. If the fine mesh was built by *refining* a coarse one, those coarser meshes already exist, and the transfers follow from the refinement relation: each fine node either coincides with a coarse node or lies inside a known coarse cell. In PETSc this is `pc_type=mg`, and run as a *full* multigrid cycle — starting by solving entirely on the coarse level on the coarsest level and working to finer and

finer renditions of the solution (cycling up and down through the meshes as it goes to accelerate removing the errors). We'll refer to this as FMG.

The difference matters when the operator's connectivity is a poor guide to the geometry, which happens when there are jumps or steep gradients in material properties. The algebraic method sees a matrix whose couplings are dominated by the stiff region and groups unknowns accordingly; the geometric method just uses the grids it has been given. It's probably not obvious which of these is a better choice so let's take a look and see. But first we'll need to see how to switch on the geometrical, full-multigrid solvers in Underworld and that starts with a multi-resolution mesh.

3. MAKING A MESH WITH A HIERARCHY

A mesh only carries a hierarchy if it was built with one. That is the `refinement` argument:

```
mesh = uw.meshing.UnstructuredSimplexBox(cellSize=0.05, refinement=2)
len(mesh.dm_hierarchy)      # 3: the base and two refinements
```

The mesh you get back is the finest level, and the coarser ones are stored within it as PETSc `DMPlex` objects in `mesh.dm_hierarchy`. They are what the preconditioner uses. They are not Underworld meshes: there is no `Mesh` object for a coarse level, and extracting one takes work: so the hierarchy is something the solver reads rather than something you need to worry about.

Build the same mesh at `cellSize=0.0125` with no refinement and you get about the same resolution with no hierarchy at all — and no geometric multigrid. The algebraic multigrid works fine on a mesh that has a hierarchy, but it does not use the coarse grid information.

The solver picks it up on its own:

```
stokes = uw.systems.Stokes(mesh)
stokes.solve()      # preconditioner defaults to "auto"
```

"auto" uses geometric multigrid when `len(mesh.dm_hierarchy) > 1` and GAMG when it does not. You can also ask directly, with `"fm"` or `"gamg"`. Two behaviours are deliberate: "auto" only ever adds geometric multigrid to an untouched configuration, so it will not overwrite preconditioner options you have set yourself; and where a request cannot be honoured, you can find the reasoning in `stokes.pc_fallbacks`. `stokes.preconditioner_settings` reports what was actually applied.

4. CURVED BOUNDARIES NEED HELP TO REFINE.

Refining a mesh puts new nodes at the midpoints of existing edges. On a straight boundary that is fine. On a curved one it is not: the coarse mesh approximates a circle by a polygon, and the midpoint of a chord lies inside the circle, not on it. Those mid points need to be moved to lie on the boundary each time the mesh is refined.

Underworld's curved meshes carry a refinement *callback* that snaps boundary-labelled nodes back onto the true surface after each refinement. For the annulus it is a few lines — take the nodes labelled `Upper` and `Lower` and rescale each to the correct radius:

```
coords[upper] *= radiusOuter / R[upper]
coords[lower] *= radiusInner / R[lower]
```

This happens automatically for the built-in meshes, so in normal use there is nothing to do. It matters if you build a mesh of your own with curved boundaries: without a callback of this kind the mesh has facets that match the coarsest mesh and do not represent the boundary at the finest resolution.

5. WHAT THE COARSE MESH LEAVES BEHIND

Because the fine mesh is made by subdividing the coarse one, the coarse mesh's own triangulation is still visible in it. Its edges survive as continuous lines across the fine mesh, and its vertices remain places where an unusual number of elements meet. The pattern is a record of how the mesh was made rather than of the problem being solved.

`mesh.relax()` moves the nodes to improve element shapes while keeping the resolution and the topology, which loosens that imprint.

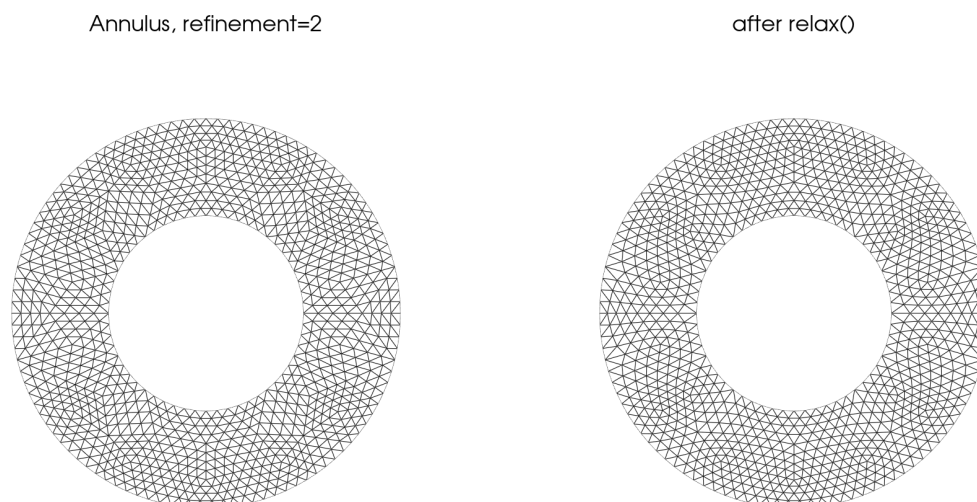


Figure 1: A twice-refined annulus, before and after `relax()`. The seams on the left are the coarse mesh's own edges, still visible after two refinements. Median element quality goes from 0.940 to 0.972 and the tenth percentile from 0.839 to 0.927; the worst single cell goes the other way, 0.827 to 0.763, because the mover minimises a global energy and will trade a few cells for many. Every node that began on a bounding circle is still exactly on it: relaxation moves interior coordinates only.

Because relaxation moves coordinates but not topology, the hierarchy survives which is the subject of a [related technical note](#).

6. WHEN THE CHOICE MATTERS

The test is SolKz, a standard Stokes benchmark on the unit box: viscosity varying exponentially with depth, $\eta = e^{2Bz}$, forced by $\mathbf{f} = (0, \sin(m\pi z) \cos(n\pi x))$ with $n = 3, m = 2$, free slip on all four walls, Taylor–Hood P_2 – P_1 elements. The viscosity contrast across the box is e^{2B} . SolKz has a closed-form solution, but nothing here uses it: these are timings, not accuracy measurements.

Work is reported as floating-point operations counted by PETSc, per unknown and relative to the cheapest FMG run. Flops are the better measure: they do not depend on the machine, and they are deterministic — repeating a run reproduces the count exactly, where a timing moves by a few per cent. Times are given alongside, because they are what you wait for and they do not tell the same story.

These numbers required fixing Underworld's GAMG settings

Underworld configures GAMG with `pc_mg_type=additive`, where PETSc's default is `multiplicative`, and hands it a velocity operator with block size 1 on a two-component field, so it aggregates scalars rather than nodes. Together those cost GAMG a factor of seven in cycles and 3.7 in arithmetic. Both are corrected here — the shipped script does it explicitly — because a comparison against that configuration measures our settings rather than the method. See [underworld3#579](#).

Cost against viscosity contrast, at 11 727 unknowns:

preconditioner	viscosity contrast	relative work per unknown	relative time per unknown
FMG	10^0	1.0	1.0
FMG	10^2	1.3	1.1
FMG	10^4	1.6	1.3
FMG	10^6	3.8	2.7
GAMG	10^0	2.5	1.4
GAMG	10^2	3.7	1.8
GAMG	10^4	4.5	2.1
GAMG	10^6	9.1	4.1

Both handle the whole range. FMG does between two and three times less arithmetic than GAMG at every contrast, and the ratio is stable — neither degrades relative to the other as the viscosity structure gets harder.

Cost against problem size, at constant viscosity:

preconditioner	unknowns	relative work per unknown	relative time per unknown	Gflop/s
FMG	2 947	1.0	1.0	2.2
FMG	11 727	1.9	1.3	3.3
FMG	46 783	2.6	1.6	3.7
FMG	186 879	2.9	1.9	3.4
FMG	747 007	3.0	2.3	3.0
GAMG	2 947	3.3	1.5	4.9
GAMG	11 727	4.7	1.8	5.7
GAMG	46 783	4.9	2.0	5.6
GAMG	186 879	5.8	2.2	5.8

Both flatten. Step by step, FMG's flop count goes as $N^{1.46}$, $N^{1.23}$, $N^{1.08}$, $N^{1.03}$; GAMG's as $N^{1.25}$, $N^{1.04}$, $N^{1.11}$. Both are converging on linear, which is what multigrid of either kind is supposed to do and what a correctly configured algebraic method delivers as well as a geometric one.

The cycle counts say the same thing. FMG converges the velocity block in three cycles at every size; GAMG takes 26–32, then 34–47, then 33–49 — ten times as many, but not a growing number. (The Schur factorisation invokes the velocity solve sixteen times per Stokes solve, the same for both, so that factor cancels.)

So the geometric hierarchy is worth something, and it is worth roughly a factor of two: FMG does about half GAMG's arithmetic on this problem, consistently, and holds that through six orders of viscosity contrast and a 250-fold growth in problem size. It is not the order-of-magnitude difference it appears to be against Underworld's shipped GAMG settings.

In wall clock the advantage is smaller still — 2.3 against 2.2 at the largest size, which is no advantage at all — and the last column says why. GAMG runs at 5–6 Gflop/s where FMG runs at 2–4, and FMG's rate *falls* at the largest size as the problem outgrows cache.

This is the classic dilemma of an efficient algorithm. Geometric multigrid on an unstructured hierarchy does less arithmetic, but in a shape the processor dislikes: sparse, indirect, with small coarse levels that cannot fill a pipeline. Algebraic multigrid does more arithmetic in denser, more regular operators that run closer to peak. Doing half the work and getting none of the time back is a fair summary of where that leaves us on this hardware, and it is the reason to keep both measures in view — flops tell you about the method, seconds tell you about the machine, and here they disagree.

What this comparison does not settle

The solver tolerance is held fixed as the mesh refines. That is the usual way to show a multigrid scaling, and it is a choice: a finer mesh has a smaller discretisation error, so an argument can be made for tightening the solver tolerance alongside it, which would make the work per unknown grow. The table answers “what does it cost to solve this system”, not “what does it cost to reach the accuracy the mesh can support”.

SolKz is also a smooth problem. Viscosity that varies smoothly everywhere is what algebraic coarsening reads best; structure that is concentrated rather than distributed is harder for it, and that case is not measured here.

7. USING IT

```
# Build the hierarchy at mesh construction: base + two refinements.
mesh = uw.meshing.UnstructuredSimplexBox(cellSize=0.05, refinement=2)

stokes = uw.systems.Stokes(mesh)
stokes.preconditioner = "auto"          # FMG when a hierarchy exists
stokes.solve()

print(len(mesh.dm_hierarchy))           # how many levels there are
print(stokes.preconditioner_settings)    # what was applied
print(stokes.pc_fallbacks)               # anything declined, and why
```

If a solve is slow and the mesh has no hierarchy, this is the first thing to change: rebuild the mesh with refinement rather than at a fine cellSize, and the same resolution arrives with the coarse levels attached. Just be aware that the cell-size determines the coarsest mesh and is refined (by a factor of 2) each time.